Interpreter

Interpreter · Pricing-rule DSL: base + percent + flat discounts · Build with Waqar

Today's Hook

- Today's force: Pricing-rule DSL: base + percent + flat discounts
- We earn Interpreter by feeling the naive pain first.
- Demo main prints a real JDK 25 trace (happy + failure).
- Key types: PricingDsl, Rule, Base, PercentOff

The Problem

- Pricing-rule DSL: Base(sku) → PercentOff → FlatOff → Stack.
- Context map supplies SKU prices; unknown sku / bad percent must fail.
- Naive stringly codes (PCT10/FLAT200) cannot compose safely.
- Success: 10% then $2 off of 2000 → 1600; Stack adds ADDON.

Structure



Interpreter: Pricing-rule DSL: base + percent + flat discounts

Real-World Scenario

- Running example: Pricing-rule DSL: base + percent + flat discounts
- Key types: PricingDsl, Rule, Base, PercentOff
- Happy-path stdout cue: 1600
- Failure-path stdout cue: caught=unknown sku NOPE

Naive Code

```java
package com.buildwithwaqar.patterns.gof.behavioral.interpreter.problem;

import java.util.Map;

/** Naive: stringly if/else discount codes — untestable composition. */
public final class NaivePricing {
    public static int price(String code, Map<String, Integer> prices) {
        int base = prices.getOrDefault("SKU", 0);
        if ("PCT10".equals(code)) return base * 90 / 100;
        if ("FLAT200".equals(code)) return Math.max(0, base - 200);
        if ("STACK".equals(code)) return Math.max(0, base * 90 / 100 - 200);
        throw new IllegalArgumentException("unknown " + code);
    }
}
```

Why Naive Hurts

- Naive: stringly if/else discount codes — untestable composition.
- Without Interpreter, call sites absorb the wrong details.
- Failure mode to show on camera: caught=unknown sku NOPE
- New requirements force shotgun edits across the naive code.

Solution

```java
package com.buildwithwaqar.patterns.gof.behavioral.interpreter.problem;

import java.util.Map;

/** Pricing-rule DSL: Base / PercentOff / FlatOff / Stack as an Expression AST. */
public final class PricingDsl {
    private PricingDsl() {}
    public sealed interface Rule permits Base, PercentOff, FlatOff, Stack {
        int apply(Map<String, Integer> ctx);
    }
    public record Base(String skuKey) implements Rule {
        public int apply(Map<String, Integer> ctx) {
            Integer v = ctx.get(skuKey);
            if (v == null) throw new IllegalArgumentException("unknown sku " + skuKey);
            return v;
        }
    }
    public record PercentOff(Rule inner, int percent) implements Rule {
        public int apply(Map<String, Integer> ctx) {
            if (percent < 0 || percent > 100) throw new IllegalArgumentException("bad percent");
            return inner.apply(ctx) * (100 - percent) / 100;
        }
```

Solution (2)

```
    }
    public record FlatOff(Rule inner, int cents) implements Rule {
        public int apply(Map<String, Integer> ctx) {
            return Math.max(0, inner.apply(ctx) - cents);
        }
    }
    public record Stack(Rule left, Rule right) implements Rule {
        public int apply(Map<String, Integer> ctx) { return left.apply(ctx) + right.apply(ctx); }
    }
}
```

Runnable main

```java
package com.buildwithwaqar.patterns.gof.behavioral.interpreter.problem;

import java.util.Map;

public final class ProblemDemo {
    public static void main(String[] args) {
        System.out.println("=== Interpreter: pricing-rule DSL ===");
        var ctx = Map.of("SKU", 2000, "ADDON", 500);
        System.out.println("--- naive ---");
        System.out.println(NaivePricing.price("STACK", ctx));
        System.out.println("--- DSL AST ---");
        PricingDsl.Rule rule = new PricingDsl.FlatOff(
                new PricingDsl.PercentOff(new PricingDsl.Base("SKU"), 10), 200);
        System.out.println("price=" + rule.apply(ctx));
        PricingDsl.Rule stack = new PricingDsl.Stack(rule, new PricingDsl.Base("ADDON"));
        System.out.println("cart=" + stack.apply(ctx));
        try {
            new PricingDsl.Base("NOPE").apply(ctx);
        } catch (IllegalArgumentException e) {
            System.out.println("caught=" + e.getMessage());
        }
    }
```

Runnable main (2)

```
}
```

Walkthrough Output

```
=== Interpreter: pricing-rule DSL ===
--- naive ---
1600
--- DSL AST ---
price=1600
cart=2100
caught=unknown sku NOPE
```

Step-by-Step Trace

- Match stdout to stages — look for: 1600
- Then the injected failure: caught=unknown sku NOPE
- Re-run: java -cp ... 17-interpreter ProblemDemo (see episode folder).
- Pause the video and predict the next printed line.

Modern Java 25 / Zazr

```java
package com.buildwithwaqar.patterns.gof.behavioral.interpreter.problem;

import java.util.Map;

/** Pricing-rule DSL: Base / PercentOff / FlatOff / Stack as an Expression AST. */
public final class PricingDsl {
    private PricingDsl() {}
    public sealed interface Rule permits Base, PercentOff, FlatOff, Stack {
        int apply(Map<String, Integer> ctx);
    }
    public record Base(String skuKey) implements Rule {
        public int apply(Map<String, Integer> ctx) {
            Integer v = ctx.get(skuKey);
            if (v == null) throw new IllegalArgumentException("unknown sku " + skuKey);
            return v;
        }
    }
    public record PercentOff(Rule inner, int percent) implements Rule {
        public int apply(Map<String, Integer> ctx) {
            if (percent < 0 || percent > 100) throw new IllegalArgumentException("bad percent");
            return inner.apply(ctx) * (100 - percent) / 100;
        }
```

Modern Java 25 / Zazr (2)

```
}
public record FlatOff(Rule inner, int cents) implements Rule {
    public int apply(Map<String, Integer> ctx) {
        return Math.max(0, inner.apply(ctx) - cents);
    }
}
public record Stack(Rule left, Rule right) implements Rule {
```

Trade-offs

- Use Interpreter when «Pricing-rule DSL: base + percent + flat discounts» repeats and a simpler feature does not erase it.
- Refuse extra types when a sealed switch or a function value already suffices.
- Teaching demos omit production metrics, retries, and multi-region clocks.
- Interview soundbite: state the force behind Interpreter in product language.

Related Patterns

- Name the pattern usually confused with Interpreter and separate the forces.
- Sketch the roles for «Pricing-rule DSL: base + percent + flat discounts» from memory in sixty seconds.
- Point to the next episode in this season on the playlist map.
- Link this episode's series/ folder in the YouTube description.

Recap

- Problem: Pricing-rule DSL: base + percent + flat discounts
- Naive pain → Interpreter boundary → Demo stdout.
- Remember the failure cue: caught=unknown sku NOPE
- Break a ProblemTest assertion, watch it fail, restore it.